

Timing constraints and real-time CPU scheduling

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Recap: von Neumann architecture

- Instructions are fetched from ROM on the same bus as data fetched from RAM
- Software is an ordered sequence of very specific instructions
- The processor executes the software by retrieving and interpreting these instructions one at a time

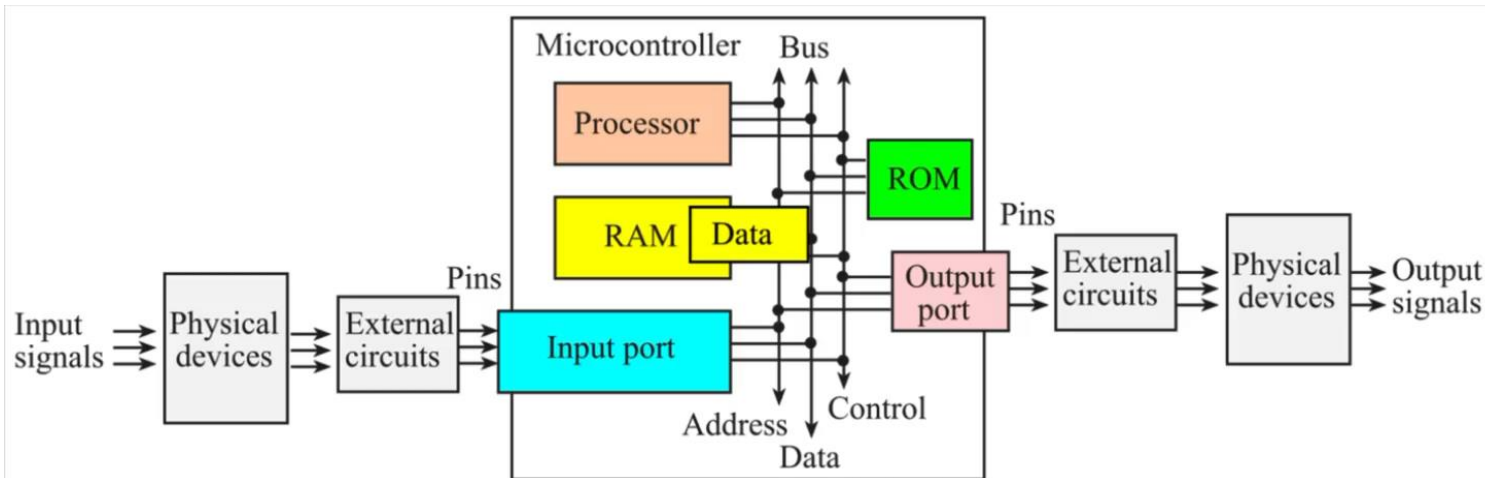


Figure 1.1.1. The basic components of a von Neumann computer including processor, memory and I/O connected with by a single bus.

Recap: Harvard architecture

- The processor can simultaneously fetch an instruction out of flash ROM using the ICode bus while it writes data into RAM using the system bus

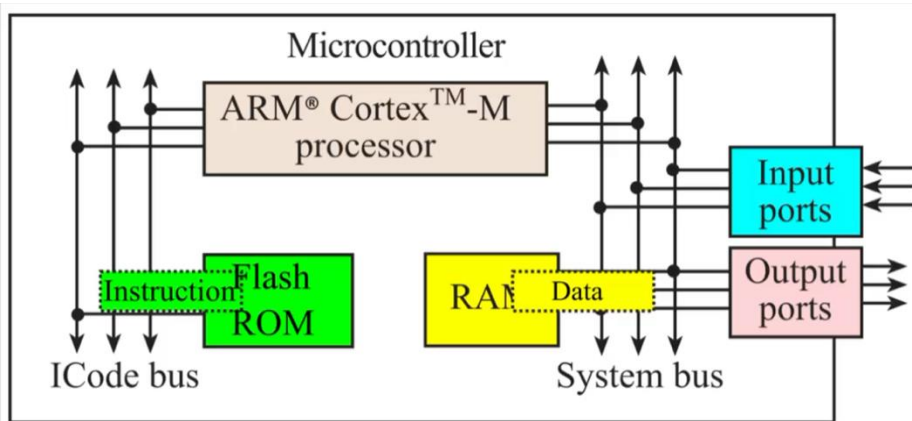


Figure 1.1.2. Harvard architecture of an ARM Cortex-M4. The three signals on each bus are address, data, control. If all these components exist on a single chip, it is called a microcontroller.

AD HOC DESIGN FAILS REAL-TIME SYSTEMS

- Ad hoc approaches rely on experience and heuristics: low-level timer code, optimization hacks, and informal testing
- Such approaches are difficult to verify for timing and memory constraints, and are fragile under rare but possible conditions
- Systematic design using formal models enables predictability, maintainability, and verifiable correctness

THE PATRIOT MISSILE: 343 MS DELAY, 28 LIVES LOST

- A software bug caused the Patriot missile's real-time clock to accumulate a 57-microsecond delay per minute of operation
- After 100 continuous hours of operation, the total accumulated error was 343 ms, producing a trajectory prediction error of 687 meters
- The system classified an incoming Scud missile as a false alarm; the missile struck Dhahran, Saudi Arabia on February 25, 1991

TIMING BUGS IN SPACE: SHUTTLE SYNCHRONIZATION AND MARS PRIORITY INVERSION

- Space Shuttle (1981): A timing bug from transient CPU overload during initialization caused a 1-in-67 chance of pushing a redundant processor out of synchronization; discovered only after intensive post-hoc analysis
- Mars Pathfinder (1997): A priority inversion bug caused the spacecraft's computer to reset repeatedly, nearly ending the mission; the root cause was a low-priority task holding a mutex needed by a high-priority task
- Both failures highlight that even intensively tested systems can harbor latent timing defects that only manifest under specific conditions

REAL-TIME CORRECTNESS DEPENDS ON BOTH VALUE AND TIME

- A real-time system is one in which the correctness of a result depends not only on its logical value but also on the time at which it is delivered
- Three flavors: Hard real-time (deadline miss = system failure), Firm real-time (result has no utility after deadline, but occasional misses are tolerable), and Soft real-time (utility degrades with latency, but system continues)
- "Real-time" does not mean "fast" — it means consistently on time; a slow but predictable system can be real-time, while a fast but jittery system may not be

SCHEDULING ALLOCATES PROCESSOR TIME TO SATISFY TIMING CONSTRAINTS

- The scheduling problem: allocate processor time to a set of tasks such that all critical timing constraints are met
- The problem has many variants: static vs. dynamic task sets, single vs. multiprocessor, periodic vs. aperiodic tasks, preemptive vs. non-preemptive, dependent vs. independent tasks
- The choice of scheduling algorithm profoundly affects whether a given task set can meet its deadlines, even when total CPU utilization is well below 100%

TIMING CONSTRAINTS GOVERN BOTH SYSTEM BEHAVIOR AND USER INTERACTION

- Performance constraints set limits on the response time of the system (e.g., the system must respond to a sensor event within 10 ms)
- Behavioral constraints govern the rate at which the environment supplies stimuli to the system (e.g., a user must dial the next digit within 20 seconds)
- Three fundamental types: Maximum (no more than t time between events), Minimum (no less than t time between events), and Durational (an event must persist for exactly t time)

FOUR COMBINATIONS OF STIMULUS-RESPONSE TIMING CONSTRAINTS

- S-S (Stimulus-Stimulus): a maximum or minimum time between two environmental inputs (e.g., "the second digit must be dialed within 20 s of the first")
- S-R (Stimulus-Response): a maximum time between an input and the system's response — the most common performance requirement (e.g., "dial tone within 2 s of lifting the receiver")
- R-R (Response-Response): a maximum time between two system outputs (e.g., "ringback tone within 0.5 s of ring tone"); R-S (Response-Stimulus): a maximum time between a system output and the next environmental input

THE PERIODIC TASK MODEL: $\Gamma = \{T_1, T_2, \dots, T_n\}$

- A task set Γ consists of n independent periodic tasks; task τ_i has period T_i , worst-case execution time C_i , and relative deadline D_i
- The j -th instance of task τ_i is denoted $\tau_{i,j}$; the phase R_i is the arrival time of the first instance $\tau_{i,1}$ (unknown in the non-concrete case)
- Key simplifying assumptions: tasks are fully preemptible, all OS overheads are zero, and in the basic model $D_i = T_i$

UTILIZATION $U = \sum C_i/T_i$ MUST NOT EXCEED 1 — BUT THAT IS NOT SUFFICIENT

- The CPU utilization factor $U = \sum_i C_i/T_i$ measures the fraction of processor time demanded by the task set; $U > 1$ means the system is definitely not schedulable
- $U \leq 1$ is necessary but not sufficient for schedulability: tasks may still miss deadlines due to phase interactions even when $U < 1$
- Example: Task 1 with $T_1 = 50$ ms, $C_1 = 25$ ms ($U_1 = 50\%$) and Task 2 with $T_2 = 100$ ms, $C_2 = 40$ ms ($U_2 = 40\%$) give $U = 90\%$ — but whether this is schedulable depends on the scheduling algorithm

THE CRITICAL INSTANT: WHEN ALL HIGHER-PRIORITY TASKS ARRIVE SIMULTANEOUSLY

- The critical instant for task τ_i is the time at which a request for τ_i will experience the maximum response time
- Theorem (Liu & Layland, 1973): The critical instant for any task occurs when it is requested simultaneously with all higher-priority tasks
- The critical time zone is the interval from the critical instant to the deadline of the corresponding request; if all tasks meet their deadlines at their critical instants, the scheduling algorithm is feasible

EMBEDDED OS SCHEDULING PRIMITIVES: THREADS, EVENT HANDLERS, AND COROUTINES

- Embedded OSes support multiple concurrency abstractions: threads (separate stacks, preemptive), event handlers (single stack, run-to-completion), and coroutines/protothreads (cooperative, single stack)
- The atomic unit of scheduling may be a thread, event handler, or coroutine; the choice affects both scheduling overhead and the complexity of timing analysis
- Key OS properties relevant to scheduling: single vs. separate address spaces, kernel-user mode protection, and whether the task set is static or dynamic

STATIC VS. DYNAMIC, PREEMPTIVE VS. NON-PREEMPTIVE SCHEDULING

- Static (offline) scheduling: the schedule is computed at design time for all possible task instances; optimal if feasible, but inflexible and computationally expensive to construct
- Dynamic (online) scheduling: priorities are computed at runtime based on current task state; more flexible but incurs runtime overhead and may be harder to analyze
- Preemptive scheduling allows a higher-priority task to interrupt a running lower-priority task; non-preemptive scheduling requires a task to run to completion, simplifying analysis but potentially causing priority inversion

PRIORITY ASSIGNMENT IS THE CORE OF PREEMPTIVE REAL-TIME SCHEDULING

- In priority-driven scheduling, each task is assigned a priority; at any instant, the highest-priority ready task executes and preempts any lower-priority running task
- Static (fixed) priority: priorities are assigned once at design time and never change (e.g., Rate-Monotonic, Deadline-Monotonic)
- Dynamic priority: priorities change at runtime, typically based on current deadline or laxity (e.g., Earliest Deadline First, Least Laxity First)

WRONG PRIORITY ASSIGNMENT CAN CAUSE DEADLINE MISSES AT 90% UTILIZATION

- Consider: Task 1 ($T_1 = 50$ ms, $C_1 = 25$ ms) and Task 2 ($T_2 = 100$ ms, $C_2 = 40$ ms); $U = 90\%$
- If Task 2 is given higher priority: at $t=0$, Task 2 runs for 40 ms, then Task 1 runs from $t=40$ to $t=50$ — Task 1 meets its deadline at $t=50$
- But at $t=100$: Task 1 and Task 2 both arrive; Task 2 runs 40 ms (to $t=140$), then Task 1 must finish by $t=100$ — Task 1 misses its deadline!

WCET ANALYSIS IS ESSENTIAL: SCHEDULABILITY GUARANTEES REQUIRE TIGHT BOUNDS

- The worst-case execution time (WCET) of a task is the maximum time it could take to execute on a specific hardware platform, over all possible inputs and initial states
- WCET analysis methods: measurement-based (simple but may underestimate), static analysis (sound but potentially pessimistic), and hybrid approaches
- Tight WCET bounds are essential: overestimating WCET leads to unnecessary CPU reservation; underestimating leads to deadline misses that the schedulability analysis cannot predict

RATE-MONOTONIC: ASSIGN HIGHER PRIORITY TO FASTER TASKS

- Rate-Monotonic (RM) priority assignment: task τ_i receives higher priority than τ_j if and only if $T_i < T_j$ (shorter period $\rightarrow$ higher priority)
- Theorem (Liu & Layland, 1973): RM is optimal among all fixed-priority scheduling algorithms — if any fixed-priority assignment can schedule a task set, RM can schedule it
- RM is the most widely used scheduling algorithm in real-time embedded systems due to its simplicity, optimality, and well-understood schedulability analysis

RM GUARANTEES SCHEDULABILITY IF $U \leq N(2^{1/N}) - 1$

- Theorem 3 (Liu & Layland): A set of n periodic tasks scheduled by RM will always meet all deadlines if $U = \sum C_i/T_i \leq n(2^{1/n}) - 1$
- As $n \rightarrow \infty$, the bound converges to $\ln 2 \approx 0.693$; for $n = 1$ the bound is 1.0, for $n = 2$ it is 0.828, for $n = 3$ it is 0.779
- This is a sufficient condition, not a necessary one: task sets with $U > n(2^{1/n}) - 1$ may still be schedulable under RM — they require more detailed analysis

SCHEDULABILITY TEST: COMPARE U TO THE RM BOUND

- Example 1: τ_1 ($C_1=20$, $T_1=100$), τ_2 ($C_2=30$, $T_2=145$); $U = 0.41 \leq 2(2^{1/2}-1) = 0.828 \rightarrow$ Schedulable
- Example 2: τ_1 ($C_1=20$, $T_1=100$), τ_2 ($C_2=30$, $T_2=145$), τ_3 ($C_3=68$, $T_3=150$); $U = 0.86 > 3(2^{1/3}-1) = 0.779$
 $\rightarrow$ Cannot determine from bound alone; must apply response-time analysis
- The utilization bound is a quick first-pass test; if U exceeds the bound, more detailed analysis (Theorem 1 / response-time analysis) is needed

RESPONSE-TIME ANALYSIS PROVIDES AN EXACT SCHEDULABILITY TEST

- For the highest-priority task τ_1 , the worst-case response time $R_1 = C_1$ (no interference from other tasks)
- For lower-priority task τ_i , the response time $R_i = C_i + I_i$, where I_i is the total interference from all higher-priority tasks during the interval $[0, R_i]$
- The interference from higher-priority task τ_j during R_i is $\lceil R_i/T_j \rceil \times C_j$ (the number of times τ_j is released during R_i , times its execution time)

$$R_i = C_i + \sum_{j \in \text{HP}(i)} \lceil R_i / T_j \rceil \times C_j \text{ — SOLVED ITERATIVELY}$$

- The response time equation $R_i = C_i + \sum_{j \in \text{hp}(i)} \lceil R_i / T_j \rceil \times C_j$ is a fixed-point equation, where $\text{hp}(i)$ is the set of tasks with higher priority than τ_i
- The iteration starts with $R_i^0 = C_i$ and computes $R_i^{n+1} = C_i + \sum_{j \in \text{hp}(i)} \lceil R_i^n / T_j \rceil \times C_j$ until convergence ($R_i^{n+1} = R_i^n$)
- If the converged value $R_i \leq D_i$, task τ_i meets its deadline; if $R_i > D_i$ (or $R_i > T_i$), the task misses its deadline

EXAMPLE: T_3 RESPONSE TIME WITH THREE TASKS

- Task set: τ_1 ($C_1=20$, $T_1=100$), τ_2 ($C_2=30$, $T_2=145$), τ_3 ($C_3=68$, $T_3=150$, $D_3=150$); RM priority: $\tau_1 > \tau_2 > \tau_3$
- Initial estimate: $R_3^0 = C_3 = 68$; First iteration: $R_3^1 = 68 + \lceil 68/100 \rceil \times 20 + \lceil 68/145 \rceil \times 30 = 68 + 20 + 30 = 118$
- Second iteration: $R_3^2 = 68 + \lceil 118/100 \rceil \times 20 + \lceil 118/145 \rceil \times 30 = 68 + 40 + 30 = 138$; Third iteration: $R_3^3 = 68 + \lceil 138/100 \rceil \times 20 + \lceil 138/145 \rceil \times 30 = 68 + 40 + 30 = 138 \rightarrow$ Converged; $R_3 = 138 \leq D_3 = 150 \rightarrow$ Schedulable!

THE HYPERBOLIC BOUND: $\prod (U_i + 1) \leq 2$ IS LESS PESSIMISTIC THAN THE UTILIZATION BOUND

- The Hyperbolic Bound (Bini, Buttazzo & Buttazzo, 2003): a task set is schedulable under RM if $\prod_i (U_i + 1) \leq 2$, where $U_i = C_i/T_i$
- This bound has the same computational complexity as the utilization bound ($O(n)$) but is less pessimistic — it accepts more task sets as provably schedulable
- It is the tightest possible bound expressible using only individual task utilization factors, making it the best simple sufficient test for RM schedulability

RM CANNOT FULLY UTILIZE THE CPU — THE 69.3% CEILING

- Even though RM is optimal among fixed-priority algorithms, it has a worst-case schedulable utilization bound that converges to $\ln 2 \approx 69.3\%$ as the number of tasks grows
- A task set with 70% CPU utilization may miss deadlines under RM — this is not a bug but a fundamental limitation of fixed-priority scheduling
- Example: $T_1=50, C_1=25, T_2=75, C_2=30$; $U = 50/50 + 30/75 = 90\%$ — Task 2 misses its deadline under RM because of the near-harmonic period relationship

HARMONIC PERIOD SETS ENABLE 100% CPU UTILIZATION UNDER RATE-MONOTONIC SCHEDULING

- A period set is harmonic if every longer period is an integer multiple of every shorter period (e.g., {5, 10, 50, 100} is harmonic; {3, 5, 7} is not)
- For harmonic period sets, the RM utilization bound is 1.0 — the CPU can be fully utilized while guaranteeing all deadlines
- Practical recommendation: design task periods as harmonic multiples (e.g., 1, 10, 100, 1000 ms) to maximize CPU utilization while retaining the simplicity of RM scheduling

DEADLINE MONOTONIC: OPTIMAL WHEN DEADLINES DIFFER FROM PERIODS

- Deadline Monotonic (DM) priority assignment: task τ_i receives higher priority than τ_j if $D_i < D_j$ (shorter relative deadline $\rightarrow$ higher priority); when $D_i = T_i$, DM reduces to RM
- DM is optimal among fixed-priority algorithms for the constrained deadline case ($C_i \leq D_i \leq T_i$): if any fixed-priority assignment can schedule the task set, DM can
- Example where RM fails but DM succeeds: τ_1 ($C_1=1, T_1=10, D_1=5$), τ_2 ($C_2=4, T_2=10, D_2=10$); RM assigns equal priority (same period), but DM correctly gives τ_1 higher priority due to shorter deadline

EARLIEST DEADLINE FIRST: ALWAYS RUN THE TASK WITH THE NEAREST DEADLINE

- EDF is a dynamic-priority scheduling algorithm: at each scheduling decision point, the task with the earliest absolute deadline among all ready tasks is selected for execution
- EDF is optimal for uniprocessor scheduling: if any scheduling algorithm can meet all deadlines for a given task set, EDF can — it achieves up to 100% CPU utilization
- Unlike RM, EDF does not require periodic tasks or fixed period-to-deadline relationships; it handles arbitrary deadline constraints

EDF IS SCHEDULABLE IF AND ONLY IF $U = \sum C_i/T_i \leq 1$

- For periodic tasks with deadlines equal to periods, EDF is schedulable if and only if $U = \sum C_i/T_i \leq 1$ — a necessary and sufficient condition
- This is a dramatic improvement over RM, which only guarantees schedulability for $U \leq n(2^{1/n} - 1) \approx 0.693$ for large n
- For tasks with deadlines different from periods ($D_i \leq T_i$), the schedulability condition becomes $\sum C_i/D_i \leq 1$

EDF ACHIEVES HIGHER UTILIZATION BUT RM IS SIMPLER AND MORE PREDICTABLE

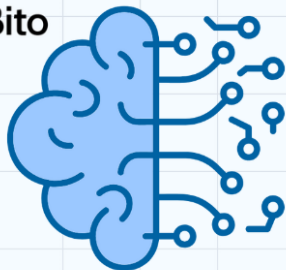
- EDF advantage: achieves 100% CPU utilization (vs. RM's ~69.3% for arbitrary periods); handles arbitrary deadline constraints; optimal for uniprocessor
- RM advantage: static priorities are simpler to implement and analyze; predictable behavior under overload (highest-priority tasks always meet deadlines); widely supported by commercial RTOSes
- Practical recommendation: use RM with harmonic periods for most embedded control systems; consider EDF when CPU utilization must exceed 69.3% and dynamic priority overhead is acceptable

LEAST LAXITY FIRST: SCHEDULE THE TASK WITH THE LEAST SLACK TIME

- Laxity (slack) of task τ_i at time t : $l_i(t) = d_i - t - c_i(t)$, where d_i is the absolute deadline and $c_i(t)$ is the remaining execution time
- Least Laxity First (LLF) always runs the task with the smallest laxity; like EDF, it is optimal for uniprocessor scheduling
- LLF advantage over EDF: laxity accounts for remaining execution time, making it more informative for tasks of mixed sizes; LLF may provide more graceful degradation under overload

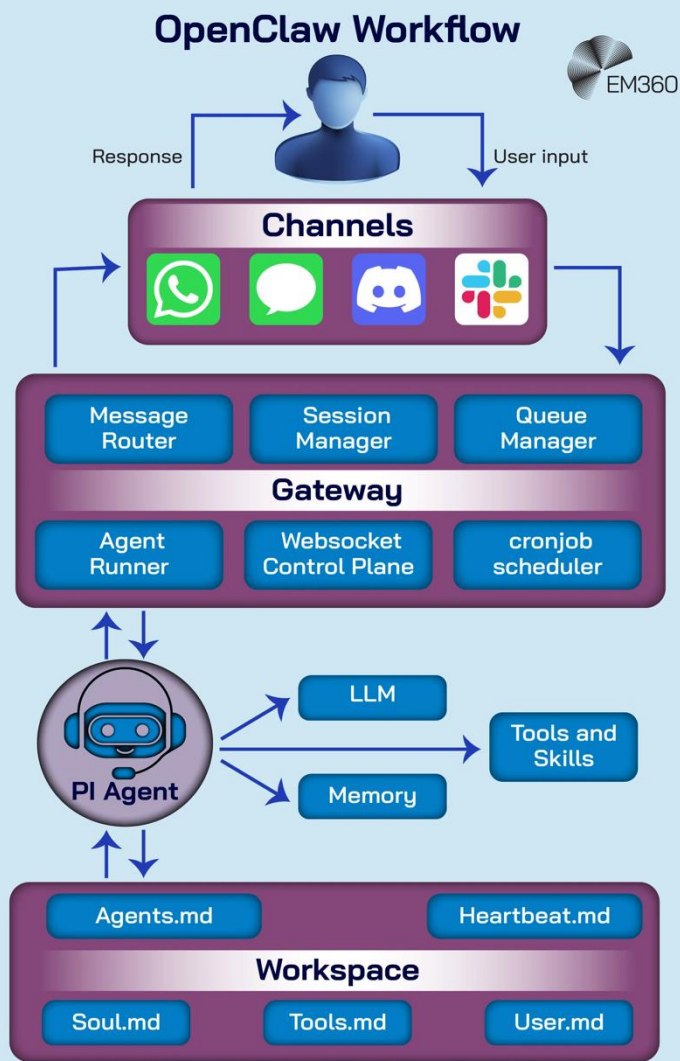
LLM vs agent

 Bito



**AI Agent
VS
Large Language Model**

OpenClaw



Installation prerequisites

- A virtual machine: 阿里云 or others (Ali cloud 1-month trial)
- Chat channel: 飞书 or others
- Coding plan: GLM-4.7 or others (GLM-4.7 free token)
- Search api: Tavily (free researcher plan)
- Reference: OpenClaw website: <https://openclaw.ai>

EDF UNDER OVERLOAD: CASCADING FAILURES VS. RM'S GRACEFUL DEGRADATION

- Under overload ($U > 1$), EDF cannot guarantee any deadlines; it tends to cause many tasks to miss their deadlines simultaneously as the system falls further behind
- RM under overload: the highest-priority (shortest-period) tasks continue to meet their deadlines; only the lowest-priority tasks miss deadlines — a more predictable degradation pattern
- Overload management strategies for EDF: admission control (reject new tasks when U would exceed a threshold), task skipping (skip low-value task instances), and elastic scheduling (adjust periods dynamically)

IMPLEMENTING EDF: PRIORITY QUEUES, DEADLINE TRACKING, AND SCHEDULING OVERHEAD

- EDF requires maintaining a priority queue ordered by absolute deadline; at each scheduling point, the task with the smallest absolute deadline is selected
- Absolute deadline must be tracked for each task instance: $d_{i,j} = r_{i,j} + D_i$, where $r_{i,j}$ is the release time of the j -th instance of task τ_i
- Scheduling overhead: EDF requires $O(\log n)$ time per scheduling decision (priority queue operations); RM requires $O(1)$ time (simple priority comparison)

EDF NATURALLY HANDLES APERIODIC AND SPORADIC TASKS WITH ARBITRARY DEADLINES

- Aperiodic tasks have unpredictable arrival times and may have soft deadlines; sporadic tasks have unpredictable arrivals but hard deadlines and a minimum inter-arrival time
- EDF handles aperiodic and sporadic tasks naturally: each task instance is assigned an absolute deadline, and EDF schedules the most urgent task regardless of its origin
- Admission control for sporadic tasks: when a new sporadic task arrives, check whether it can be scheduled (using a feasibility test) before admitting it; reject it if it would cause existing tasks to miss their deadlines

PROCESSOR DEMAND CRITERION: $\sum C_i \times \lfloor (L + T_i - D_i) / T_i \rfloor \leq L$ FOR ALL $L > 0$

- The processor demand criterion (Baruah et al., 1990) provides a necessary and sufficient schedulability test for EDF with arbitrary deadlines: the total processor demand in any time interval $[0, L]$ must not exceed L
- Demand function: $\text{dbf}(0, L) = \sum_i \max(0, \lfloor (L + T_i - D_i) / T_i \rfloor) \times C_i$; the task set is EDF-schedulable iff $\text{dbf}(0, L) \leq L$ for all $L > 0$
- In practice, it suffices to check the condition at all deadline points in the hyperperiod, reducing the test to a finite computation

SERVING APERIODIC TASKS WITHOUT DISRUPTING PERIODIC TASK SCHEDULABILITY

- Aperiodic tasks (e.g., user inputs, network packets) arrive unpredictably and must be served promptly, but their service must not cause periodic tasks to miss deadlines
- Naive approach: run aperiodic tasks at the lowest priority (background processing); this guarantees periodic task schedulability but gives poor aperiodic response times
- Polling server: a periodic task with period T_s and budget C_s that checks for aperiodic requests at each period; provides bounded service but wastes budget if no aperiodic tasks are pending

DEFERRABLE AND SPORADIC SERVERS PRESERVE BUDGET FOR IMPROVED APERIODIC RESPONSE

- Deferrable Server: unlike the polling server, the deferrable server preserves its budget until the end of its period; if an aperiodic task arrives, it is served immediately using the preserved budget
- Sporadic Server (Sprunt, Sha & Lehoczky, 1989): replenishes its budget only after it has been consumed, based on the time of consumption; provides better schedulability analysis than the deferrable server
- Both servers can be treated as periodic tasks in the RM schedulability analysis, allowing the impact of aperiodic service on periodic task schedulability to be quantified

CONSTANT BANDWIDTH SERVER: INTEGRATING APERIODIC SERVICE INTO EDF

- The Constant Bandwidth Server (CBS, Abeni & Buttazzo, 1998) integrates aperiodic service into EDF by assigning a virtual deadline to each aperiodic request
- CBS parameters: server utilization $U_s = Q_s/T_s$ (budget Q_s , period T_s); when an aperiodic task arrives, it is assigned deadline $d_s = r_s + T_s$, where r_s is the current time
- CBS guarantees that the total utilization of the system (periodic + aperiodic) does not exceed 1, preserving EDF schedulability for all tasks

SPORADIC TASKS: HARD DEADLINES WITH BOUNDED ARRIVAL RATE

- A sporadic task has a hard deadline and a minimum inter-arrival time (MIT): consecutive instances are separated by at least MIT time units
- Sporadic tasks can be modeled as periodic tasks with period equal to the MIT for schedulability analysis purposes: this is conservative (actual arrivals may be less frequent) but guarantees schedulability
- Admission control for sporadic tasks: when a new instance arrives, check whether it can be scheduled given the current system state; reject or defer if it would cause deadline misses

RELEASE JITTER DEGRADES SCHEDULABILITY AND MUST BE ACCOUNTED FOR

- Release jitter is the variation in the actual release time of a task instance relative to its nominal release time; it arises from interrupt latency, OS scheduling overhead, and hardware variability
- Jitter effectively increases the worst-case response time: a task that is released late has less time to meet its deadline; a task that is released early may interfere with other tasks earlier than expected
- Accounting for jitter in schedulability analysis: add the maximum jitter J_i to the response time equation: $R_i = C_i + J_i + \sum_{j \in hp(i)} \lceil (R_i + J_j) / T_j \rceil \times C_j$

WHEN DEADLINES EXCEED PERIODS: MULTIPLE ACTIVE INSTANCES

- When $D_i > T_i$, multiple instances of task τ_i may be active simultaneously: a new instance is released before the previous one has completed
- This situation requires careful management: the scheduler must track multiple instances, and the response-time analysis must account for the interference from all active instances
- Practical handling: limit the number of concurrent instances (e.g., using a semaphore), or use a more general scheduling model that explicitly handles multi-instance tasks

PRIORITY INVERSION: A LOW-PRIORITY TASK BLOCKS A HIGH-PRIORITY TASK

- Priority inversion occurs when a high-priority task is blocked by a low-priority task that holds a shared resource (mutex), and a medium-priority task preempts the low-priority task, preventing it from releasing the resource
- The duration of priority inversion is unbounded in the basic priority-driven scheduling model: the high-priority task may be blocked for an arbitrary length of time
- Classic example: Mars Pathfinder (1997) — a low-priority meteorological task held a mutex needed by a high-priority data distribution task; a medium-priority task preempted the low-priority task, causing the high-priority task to miss its deadline

PRIORITY INHERITANCE: TEMPORARILY ELEVATE THE PRIORITY OF THE BLOCKING TASK

- Priority Inheritance Protocol (PIP, Sha, Rajkumar & Lehoczky, 1990): when a high-priority task τ_H is blocked by a low-priority task τ_L holding a mutex, τ_L inherits the priority of τ_H until it releases the mutex
- PIP bounds the blocking time: a task can be blocked for at most the duration of one critical section of each lower-priority task that holds a shared resource
- PIP limitation: it does not prevent deadlock (if tasks can hold multiple mutexes simultaneously) and may cause chained blocking (a task is blocked by a chain of lower-priority tasks)

PRIORITY CEILING PROTOCOL: PREVENT DEADLOCK AND BOUND BLOCKING TO ONE CRITICAL SECTION

- Priority Ceiling Protocol (PCP, Sha, Rajkumar & Lehoczky, 1990): each mutex is assigned a priority ceiling equal to the highest priority of any task that may lock it; a task can lock a mutex only if its priority is strictly higher than the ceiling of all currently locked mutexes
- PCP guarantees: (1) a task can be blocked by at most one lower-priority task, (2) deadlock is prevented, (3) transitive blocking is prevented
- PCP is more complex to implement than PIP but provides stronger guarantees and tighter blocking time bounds

IMMEDIATE PRIORITY CEILING (STACK RESOURCE POLICY): SIMPLER AND EQUALLY EFFECTIVE

- Immediate Priority Ceiling Protocol (IPCP, also called Stack Resource Policy): when a task locks any mutex, its priority is immediately raised to the ceiling of that mutex, regardless of whether any other task is waiting
- IPCP is simpler to implement than PCP (no need to check the ceilings of all locked mutexes) and provides the same blocking time bound (at most one critical section)
- IPCP is used in the POSIX PTHREAD_PRIO_INHERIT mutex protocol and is the default priority ceiling mechanism in many RTOSes

COMPUTING BLOCKING TIME: THE KEY INPUT TO SCHEDULABILITY ANALYSIS WITH SHARED RESOURCES

- Blocking time B_i for task τ_i : the maximum time τ_i can be delayed by lower-priority tasks holding shared resources; B_i is added to the response-time equation: $R_i = C_i + B_i + \sum_{j \in \text{hp}(i)} \lceil R_i/T_j \rceil \times C_j$
- Under PIP/PCP, $B_i = \max$ over all lower-priority tasks τ_l of $(\text{usage}(\tau_l, S_k) \times C_{sk})$, where S_k is a semaphore that τ_i or a higher-priority task uses, and C_{sk} is the critical section duration
- Minimizing critical section lengths is essential: long critical sections increase blocking time and can make otherwise schedulable task sets unschedulable

RM WITH BLOCKING: $U + B/T \leq N(2^{1/N} - 1)$ FOR EACH TASK

- The RM utilization bound with blocking (Sha, Rajkumar & Lehoczky, 1990): for each task τ_k , $\sum_{i \leq k} C_i/T_i + B_k/T_k \leq k(2^{1/k} - 1)$
- Blocking time effectively reduces the available CPU capacity: a task that is blocked for B_k time units in each period has an effective utilization of $C_k/T_k + B_k/T_k$
- For harmonic periods, the bound becomes $\sum_{i \leq k} C_i/T_i + B_k/T_k \leq 1$, allowing full CPU utilization minus the blocking overhead

DEADLOCK IN REAL-TIME SYSTEMS: PREVENTION IS BETTER THAN DETECTION

- Deadlock occurs when two or more tasks are each waiting for a resource held by the other; in real-time systems, deadlock is catastrophic because it causes tasks to miss all future deadlines
- Prevention strategies: resource ordering (always acquire mutexes in a fixed global order), priority ceiling protocol (prevents deadlock by construction), and single-mutex design (avoid holding multiple mutexes simultaneously)
- Detection and recovery is generally not acceptable in hard real-time systems due to the unpredictable recovery time; prevention is the only viable approach

MULTIPROCESSOR PRIORITY INVERSION: REMOTE BLOCKING AND SPIN LOCKS

- In multiprocessor systems, a task on one processor may be blocked waiting for a resource held by a task on another processor; this is called remote blocking
- Spin locks (busy-waiting) are commonly used for short critical sections in multiprocessor systems; they avoid the overhead of blocking and context switching but waste CPU cycles
- Multiprocessor locking protocols (e.g., MPCP, FMLP) extend PIP/PCP to multiprocessor systems, providing bounded blocking times while accounting for both local and remote blocking

MULTIPROCESSOR SCHEDULING: PARTITIONED, GLOBAL, AND SEMI-PARTITIONED APPROACHES

- Partitioned scheduling: tasks are statically assigned to processors and scheduled independently on each processor; simple to implement but may waste CPU capacity due to bin-packing inefficiency
- Global scheduling: tasks can migrate between processors; a single global scheduler selects the m highest-priority tasks to run on m processors; can achieve higher utilization but is harder to analyze
- Semi-partitioned scheduling: most tasks are partitioned, but a few tasks are allowed to migrate; combines the simplicity of partitioned with the efficiency of global scheduling

THE DHALL EFFECT: GLOBAL EDF CAN MISS DEADLINES EVEN AT VERY LOW UTILIZATION

- The Dhall effect (1977): on m processors, global EDF can fail to schedule a task set with utilization arbitrarily close to 1, if one task has utilization close to 1 and many small tasks fill the remaining capacity
- Example: $m=2$ processors, one task with $C=1-\epsilon$, $T=1$ (utilization ≈ 1), and many tasks with $C=\epsilon$, $T=2\epsilon$ (utilization ≈ 0.5 each); total utilization ≈ 1 , but the large task misses its deadline
- The Dhall effect motivates the use of partitioned scheduling or specialized global algorithms (e.g., Pfair, DP-Fair) for multiprocessor real-time systems

ASSIGNING TASKS TO PROCESSORS: FIRST-FIT DECREASING AND RATE-MONOTONIC NEXT-FIT

- Task assignment in partitioned scheduling is a bin-packing problem: assign tasks to processors such that each processor's utilization does not exceed the schedulability bound
- First-Fit Decreasing (FFD): sort tasks by decreasing utilization; assign each task to the first processor that can accommodate it without exceeding the utilization bound
- Rate-Monotonic Next-Fit (RMNF): assign tasks to processors in order of decreasing period; when a processor's utilization bound is exceeded, start a new processor; achieves near-optimal processor utilization

DVFS AND REAL-TIME SCHEDULING: TRADING ENERGY FOR TIMING SLACK

- Dynamic Voltage and Frequency Scaling (DVFS) reduces processor power consumption by lowering the supply voltage and clock frequency; lower frequency increases task execution time, potentially threatening schedulability
- Real-time DVFS: if a task completes early (actual execution time $<$ WCET), the saved time can be used to run at a lower frequency in future periods, reducing energy consumption while maintaining schedulability
- Key challenge: WCET is the worst case; actual execution times are often much shorter, creating slack that can be exploited for energy savings without violating timing constraints

MIXED-CRITICALITY SYSTEMS: DIFFERENT ASSURANCE LEVELS FOR DIFFERENT TASKS

- Mixed-criticality (MC) systems contain tasks with different levels of criticality (e.g., safety-critical, mission-critical, best-effort) that must coexist on the same processor
- The Vestal model (2007): each task has multiple WCET estimates, one for each criticality level; in normal operation, low WCET estimates are used; when a high-criticality task exceeds its low WCET, the system switches to a high-criticality mode
- MC scheduling algorithms (e.g., EDF-VD by Baruah et al., 2011) provide schedulability guarantees for high-criticality tasks in all modes while maximizing the service provided to low-criticality tasks in normal mode

LINUX SCHED_DEADLINE: EDF SCHEDULING IN A GENERAL-PURPOSE OS

- Linux SCHED_DEADLINE (merged in Linux 3.14, 2014) implements EDF scheduling using the Constant Bandwidth Server framework; each task specifies its runtime (budget), deadline, and period
- SCHED_DEADLINE provides admission control: a new task is admitted only if the total utilization does not exceed a configurable threshold (default: 95% to reserve capacity for non-real-time tasks)
- SCHED_DEADLINE supports multiprocessor systems with global and partitioned scheduling modes, and integrates with the Linux power management framework for energy-efficient real-time scheduling

RTOS SELECTION: MATCHING SCHEDULING SUPPORT TO APPLICATION REQUIREMENTS

- Commercial RTOSes (VxWorks, QNX, Integrity): provide fixed-priority preemptive scheduling with priority inheritance, certified for safety-critical applications (DO-178C, IEC 61508, ISO 26262)
- Open-source RTOSes (FreeRTOS, Zephyr, RTEMS): widely used in embedded systems; support fixed-priority scheduling with configurable priority levels; RTEMS also supports EDF scheduling
- Linux with PREEMPT_RT patch: provides near-RTOS performance for soft real-time applications; SCHED_FIFO and SCHED_RR for fixed-priority, SCHED_DEADLINE for EDF

AUTOMATED SCHEDULABILITY ANALYSIS: FROM SPREADSHEETS TO FORMAL VERIFICATION

- Simple tools: spreadsheet-based utilization calculation and response-time analysis; suitable for small task sets with well-defined parameters
- Specialized tools: Cheddar (open-source), SymTA/S (Symtavision), RapiTime (Rapita Systems); provide automated schedulability analysis, WCET estimation, and visualization of scheduling timelines
- Formal verification tools: model checkers (UPPAAL, SPIN) can verify timing properties of real-time systems against formal specifications, providing the highest level of assurance

AUTOSAR: FIXED-PRIORITY SCHEDULING FOR AUTOMOTIVE EMBEDDED SYSTEMS

- AUTOSAR (AUTomotive Open System ARchitecture) is the dominant software architecture for automotive ECUs; it uses a fixed-priority preemptive OS (OSEK/VDX) with static task configuration
- AUTOSAR tasks are configured at design time with fixed periods, priorities, and deadlines; the OSEK OS provides rate-monotonic-compatible scheduling with priority ceiling protocol for resource sharing
- Automotive real-time requirements: engine control (1 ms cycle), transmission control (10 ms), body control (100 ms), infotainment (best-effort); harmonic periods are used to maximize schedulability

ARINC 653: TIME-PARTITIONED SCHEDULING FOR AVIONICS SAFETY

- ARINC 653 is the avionics application software standard for Integrated Modular Avionics (IMA); it uses a two-level scheduling hierarchy: partitions (time-triggered, statically scheduled) and processes within partitions (fixed-priority preemptive)
- Each partition is allocated a fixed time window in a static major frame; within its window, a partition runs its processes using a fixed-priority preemptive scheduler
- ARINC 653 provides temporal and spatial isolation between partitions: a fault in one partition cannot affect the timing of another partition, enabling mixed-criticality operation on a single processor

TEN RULES FOR DESIGNING SCHEDULABLE REAL-TIME SYSTEMS

- Use harmonic periods (e.g., 1, 10, 100, 1000 ms) to maximize CPU utilization under RM scheduling; assign priorities according to the RM or DM rule
- Keep critical sections short and minimize interrupt masking time; use the priority ceiling protocol to bound blocking time and prevent deadlock
- Always use worst-case execution times in schedulability analysis; measure WCET on the actual hardware with caches and interrupts enabled; add a safety margin (typically 20–30%)

REAL-TIME SCHEDULING: FROM THEORY TO PRACTICE

- Core theory: the periodic task model, rate-monotonic and EDF scheduling, response-time analysis, and the critical instant theorem provide a complete framework for designing and verifying real-time systems
- Key algorithms: RM (optimal fixed-priority, 69.3% utilization bound for arbitrary periods, 100% for harmonic), EDF (optimal dynamic-priority, 100% utilization), DM (optimal for constrained deadlines), and server algorithms for aperiodic/sporadic tasks
- Practical challenges: WCET analysis, priority inversion (solved by PIP/PCP), aperiodic task handling (solved by server algorithms), multiprocessor scheduling, power management, and mixed-criticality — all are active research areas with practical solutions